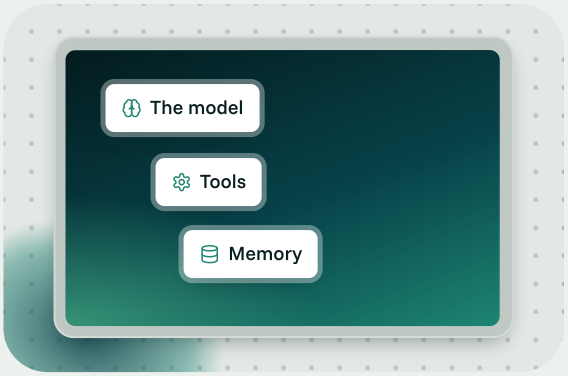
Last updated August 2026

AI Agents

How language models move from answering questions to pursuing goals, using tools, and completing multi-step tasks on their own.



15 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 4 of 10

What Is an AI Agent?

In the first three guides, we covered the building blocks. A language model predicts the next token. Prompt engineering shapes what the model produces. RAG gives the model access to external knowledge. An AI agent is what happens when you wrap all of this together and give the model the ability to act on its own.

An agent is a software system that uses a language model as its reasoning engine to pursue a goal. Instead of answering a single question and stopping, an agent receives a goal, decides what to do, takes actions, observes the results, and repeats until the goal is met or the agent determines it cannot be achieved. The model is the brain. The agent is the body around it that lets the brain interact with the world.

The distinction between a chatbot and an agent is the distinction between talking and doing. When you ask ChatGPT to summarise a document, it produces text and stops. When you give an agent the task of finding the document, summarising it, and emailing the summary to your team, it executes a sequence of steps to achieve that goal. Each step involves reasoning about what to do next, taking an action, and observing what happened.

Key principle

An agent is defined by its loop. It reasons about the current state, takes an action, observes the result, and reasons again. This loop, often called the ReAct cycle (reason, act, observe), is what separates an agent from a model. A model responds. An agent pursues.

Agents, Assistants, and Bots

These three terms get used interchangeably, but they describe different points on a spectrum of autonomy. Understanding the difference helps you choose the right architecture for a given task.

Agent	Assistant	Bot
Receives a goal and pursues it autonomously through multiple steps, deciding what to do at each point without human intervention.	Receives a request, responds, and waits for the next request. Can recommend actions, but the human decides whether to take them.	Follows predefined rules. No reasoning. Triggers produce fixed responses. Cannot adapt to unexpected input.
Books your flight, reserves the hotel, adds the trip to your calendar, and emails you the itinerary.	Suggests flights and hotels based on your preferences. You click to confirm each booking.	Sends you a reminder when check-in opens 24 hours before your flight.

The key variable is autonomy. A bot follows rules. An assistant responds to requests. An agent pursues goals. As you move from bot to assistant to agent, the system takes on more responsibility for deciding what to do next, and the human exercises less direct control over each step.

Autonomy is a trade-off. More autonomy means the agent can complete more complex tasks with less supervision, but it also means the system can go further in the wrong direction before a human catches

the error. Production agent systems almost always include guardrails that limit what the agent can do without human approval, especially for actions that are irreversible or costly.

Anatomy of an Agent

Every agent has four components that determine what it can do and how well it does it. Understanding each component helps you reason about where agents fail and how to improve them.

Pascal Academy

If your team is building agents, Pascal Academy's hands-on workshop on agent architecture walks through building a production agent end to end, from tool design to evaluation. Reach out to learn more about team training.

Ask about team training

The Model

The model is the reasoning engine. In practice, this is an LLM, often one with strong tool-use capabilities like GPT-4o, Claude 3.5, or Gemini 1.5. The model takes in the current state of the task, the history of actions taken so far, and the available tools, then produces the next action or a final answer.

The choice of model matters because agents make many sequential calls. A model that is slightly worse at reasoning produces more failed steps, and each failed step costs tokens and time. A model that is bad at tool use will produce malformed function calls, call the wrong tool, or hallucinate parameters that do not exist. Tool-use capability is arguably the single most important model quality for agentic tasks.

Tools

Tools are the functions an agent can call to interact with the world. A search tool lets it look things up. A calculator tool lets it do arithmetic reliably. An email tool lets it send messages. A database query tool lets it read and write data. Without tools, an agent is just a chatbot with extra steps. Tools are what make the agent capable of actually doing things.

Each tool is defined by a name, a description, and a schema that specifies what parameters it accepts. The model sees these definitions and decides which tool to call and with what arguments. The agent framework executes the tool call and returns the result to the model as an observation. The model then reasons about the result and decides the next step.

The quality of tool descriptions is a major factor in agent performance. If a tool description is vague, the model will call it at the wrong times or pass the wrong parameters. If two tools have overlapping functionality, the model will struggle to choose between them. Writing good tool descriptions is a form of prompt engineering applied to the function definitions themselves.

Memory

Memory lets an agent maintain context across the steps of a task and across multiple conversations. Without memory, each model call is independent and the agent has no history of what it has already tried.

Short-term memory is the conversation history within the current task. It includes the original goal, the actions taken, and the observations received. This lives in the context window and has the same limitation as any context window, it has a maximum size. Long tasks eventually exceed the window, and the agent needs a strategy for managing that, whether through summarisation, selective retention, or external storage.

Long-term memory persists across sessions. It stores facts about the user, preferences, past interactions, and learned patterns. This is typically implemented as a vector database or a key-value store that the agent can query at the start of a new task. Episodic memory, a subset of long-term memory, stores specific past interactions that the agent can recall when facing a similar situation.

Persona and Instructions

The system prompt defines the agent's role, its constraints, and how it should approach problems. A well-crafted persona tells the model what kind of reasoning to apply, what trade-offs to make, and when to ask for help. For example, a coding agent might be instructed to always run tests before declaring a task complete, to prefer simple solutions over clever ones, and to ask the user for clarification when requirements are ambiguous.

The persona also shapes how the agent communicates its reasoning. Some agent frameworks ask the model to produce a thought before each action, explaining why it chose that action. This visible reasoning, drawn from the ReAct framework, helps with debugging and gives the human confidence that the agent is on the right track.

Go deeper

The memory problem

Optional

Memory is the hardest problem in agent design. The context window is finite, but tasks can generate enormous amounts of intermediate state. An agent doing a research task might produce thousands of tokens of observations across dozens of steps. Deciding what to keep and what to discard is a judgment call, and the agent itself often has to make it. Common strategies include keeping only the most recent N steps, summarising older steps into a compressed form, and using a separate retrieval system to fetch relevant past context on demand. None of these is perfect, and the choice depends on the task.

The ReAct Loop

The ReAct framework, introduced in a 2022 paper by Yao et al., gave a name and a structure to the core loop that agents run. The name combines reasoning and acting, and the loop has three phases.

Phase	What Happens
Reason	The model looks at the goal, the actions taken so far, and the latest observation. It produces a thought about what to do next and why.
Act	The model selects a tool and produces the arguments to call it with. The agent framework executes the tool call.
Observe	The framework returns the tool's output to the model. The model incorporates this observation into its context and begins the next reasoning step.

The loop continues until the model decides the goal is achieved and produces a final answer, or until it determines the goal cannot be achieved and reports failure. Each iteration consumes tokens, so the number of iterations is a cost driver. A well-designed agent completes most tasks in 5 to 15 iterations. If an agent is running 50 iterations, something is wrong with the task framing, the tools, or the model.

What makes the ReAct loop powerful is that the reasoning is explicit. The model writes out its thought process before each action, and those thoughts become part of the context for subsequent steps. This is the same mechanism as chain-of-thought prompting from the prompt engineering guide, applied to an iterative loop. The intermediate reasoning helps the model stay on track across multiple steps rather than jumping to a conclusion after the first observation.

Planning and Task Decomposition

Simple agents receive a goal and execute the ReAct loop until the goal is met. This works for short tasks that take a few iterations. For complex tasks, the agent benefits from planning upfront, breaking the goal into sub-tasks before starting execution.

Planning is a separate model call that happens before the action loop begins. The agent receives the goal and produces a list of steps it intends to take. Each step becomes a smaller, more manageable sub-task that the agent tackles individually. If the plan says search for the company's revenue data, find the last three years of figures, compare them to industry averages, and write a summary, the agent works through each step sequentially.

The benefit of explicit planning is that it gives the agent structure. Instead of reasoning about a large goal at every step, the agent reasons about a small sub-task within the context of a known plan. This reduces the chance of the agent going off track or forgetting parts of the task.

Plans are not static. A good agent revises its plan as it gathers information. If the search results reveal that the company does not publish revenue figures, the agent updates the plan to find alternative sources rather than continuing with a plan that no longer makes sense. This adaptability is what distinguishes an agent from a fixed pipeline.

Pascal Academy

Pascal Academy's course on building AI agents includes a module on planning strategies, with hands-on exercises in task decomposition, plan revision, and handling dependencies between sub-tasks. Check the course catalogue for the next cohort.

Ask about team training

Multi-Agent Systems

A single agent can handle many tasks, but some problems benefit from multiple agents working together. A multi-agent system assigns different roles to different agents, each with its own persona, tools, and model. The agents communicate with each other to coordinate their work.

A common pattern is the researcher-writer setup. One agent gathers information and another produces the final output. The researcher agent has search tools and is optimised for finding and evaluating sources. The writer agent has a different persona focused on clarity and structure. The researcher passes its findings to the writer, who produces the document. Each agent does what it is good at.

Another pattern is the critic. After an agent produces a draft or a plan, a second agent reviews it and provides feedback. The first agent revises based on the feedback. This produces higher quality output than a single agent working alone, because the critic catches errors and oversights that the original agent missed. The trade-off is cost, because each review cycle means additional model calls.

Multi-agent systems add a coordination layer that single agents do not need. You have to decide how agents communicate, whether they work sequentially or in parallel, how to handle disagreements, and when to stop the review cycle. These are orchestration problems, and frameworks like LangGraph, CrewAI, and Google's Agent Development Kit exist to handle them.

Common Failure Modes

Why this matters

These failure modes are not theoretical. Every production agent system encounters them. The difference between a good agent and a dangerous one is not the model, it is the guardrails around the model. Budget for failure handling from the start, not as an afterthought.

Infinite Loops

The agent gets stuck repeating the same action or cycling between two actions without making progress. This happens when the model does not recognise that a previous approach failed and tries it again with slightly different phrasing. The fix is to include a maximum iteration count and to instruct the agent to try a different approach if an action fails twice.

Hallucinated Tool Calls

The model invents a tool that does not exist or passes parameters that do not match the tool's schema. This is more common with weaker models and with poorly described tools. The fix is to validate every tool call before execution and return a clear error message to the model when the call is invalid, so the model can correct itself in the next iteration.

Context Overload

As the agent takes more steps, the conversation history grows. Observations from tool calls can be verbose, especially when a search tool returns long documents. Eventually the context window fills up, the model loses track of earlier steps, and performance degrades. The fix is to summarise or truncate old observations, keeping only the information that is relevant to the remaining steps.

Cost Runaway

Each iteration of the ReAct loop consumes tokens, and a complex task with many steps can accumulate significant cost. An agent that runs 20 iterations at 2,000 tokens per call uses 40,000 tokens before

producing a final answer. If the task is not worth the cost, the agent should not have been used. Setting a cost ceiling and monitoring token usage per task is essential for production systems.

Overconfident Action

The agent takes an irreversible action, like sending an email or deleting a record, based on a flawed plan or an incorrect observation. This is the most dangerous failure mode because the consequences are real. The fix is to require human approval for irreversible actions and to run reversible actions in a dry-run mode first. The agent should never have the authority to take destructive action without a human in the loop.

Building Production Agents

Building a demo agent that works on a few examples is relatively straightforward. Building one that works reliably in production is a different challenge. The gap between demo and production is where the engineering effort lives.

Pascal Academy

Pascal Academy offers a dedicated programme on production agent deployment, covering evaluation frameworks, guardrail design, and human-in-the-loop patterns. If your organisation is moving from prototype to production, this programme is designed for your engineering team.

Ask about team training

Evaluation

As with RAG, evaluation is the highest-value investment. Build a set of 30 to 50 representative tasks with known correct outcomes. For each task, define what success looks like. Run the agent against this set and measure the percentage of tasks completed correctly, the average number of iterations, the average token cost, and the failure modes that occur. When you change the model, the tools, or the system prompt, re-run the evaluation.

Guardrails

Guardrails are the safety mechanisms that prevent the agent from doing harm. They include maximum iteration counts, cost ceilings, tool-level permissions (what each tool is allowed to do), human approval gates for irreversible actions, and output validation (checking that the agent's final answer meets quality criteria before returning it to the user). The design principle is to make it easy for the agent to do the right thing and hard for it to do the wrong thing.

Human-in-the-Loop

For tasks where errors are costly, the agent should pause and ask for human approval at key decision points. This is especially important for actions that affect real users, modify production data, or involve financial transactions. The human reviews the agent's proposed action, approves or rejects it, and the agent continues. This pattern combines the efficiency of autonomous execution with the safety of human oversight.

Use Cases

Agents are being deployed across a range of domains. The use cases below are categories, not specific products, and each one involves different trade-offs in agent design.

Agent Type	What It Does and Where the Difficulty Lives
Customer agents	Handle support queries, resolve issues, and recommend products across channels. The difficulty is handling the long tail of unusual requests and knowing when to escalate to a human. A good customer agent resolves 70 to 80% of queries autonomously and escalates the rest.
Employee agents	Simplify internal processes, answer policy questions, and manage repetitive tasks like expense reporting or onboarding. The difficulty is integrating with internal systems and maintaining access controls across different data sources.
Data agents	Analyse datasets, generate reports, and surface insights. The difficulty is ensuring the agent interprets data correctly and does not produce confident but wrong conclusions. Data agents benefit from a verification step where a second agent or a human reviews the findings.
Code agents	Generate, review, and debug code. The difficulty is that code is unforgiving, a single error breaks the build. Code agents need to run tests as part of their loop and use the test results to iterate.
Security agents	Monitor for threats, investigate incidents, and respond to attacks. The difficulty is that security involves adversarial actors who adapt their tactics. Security agents need real-time data access and the ability to act quickly, balanced against the risk of false positives causing disruption.

When Not to Use an Agent

Agents are powerful, and the temptation is to use them for everything. This is a mistake. Agents add cost, latency, and unpredictability compared to a direct model call or a simple pipeline. Before building an agent, ask whether the task actually requires autonomous decision-making.

If the task is a single question-answer interaction, use a direct model call with a good prompt. If the task is a fixed sequence of steps that never varies, build a pipeline that calls the model at each stage. If the task requires looking up information and answering from it, use RAG. Agents are the right choice when the task requires deciding what to do next based on the results of previous steps, and when the sequence of steps cannot be determined in advance.

A practical test. Can you write down the steps the agent will take before it starts? If yes, write a pipeline. If the steps depend on what the agent finds along the way, and you genuinely cannot predict them, an agent is appropriate. The value of an agent is its ability to handle the unpredictable. Using an agent for a predictable task adds cost and failure modes without adding value.

Summary

An AI agent is a system that uses a language model to pursue goals through an iterative loop of reasoning, acting, and observing. The four components, model, tools, memory, and persona, determine what the agent can do and how well it does it. The ReAct loop is the core mechanism. Planning helps with complex tasks by breaking them into sub-tasks. Multi-agent systems specialise roles for better performance on complex problems. The main failure modes are infinite loops, hallucinated tool calls, context overload, cost runaway, and overconfident irreversible actions. Production systems need evaluation sets, guardrails, and human-in-the-loop for high-stakes actions. And agents are the right tool only when the sequence of steps cannot be determined in advance.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, and agents. If you found this useful, the full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is an AI agent and how is it different from a chatbot?

An AI agent is a system that uses a language model as its reasoning engine to pursue a goal through multiple steps. A chatbot responds to a single question and stops. An agent receives a goal, decides what to do, takes actions, observes results, and repeats until the goal is met or it determines the goal cannot be achieved.

What is the ReAct loop in AI agents?

The ReAct loop is the cycle of reasoning, acting, and observing that an agent follows. The agent reasons about the current state, takes an action like searching a filing or extracting data, observes the result, and reasons again. This loop is what separates an agent from a model that simply responds to a prompt.

What AI agents does Pascal offer for investment research?

Pascal offers three core agents for institutional finance. The Generative Matrix filters and analyzes company universes. The KPI Extractor pulls tables and graphs from filings into Excel. The Research Memo Generator produces structured research reports using your fund's templates. All three run inside your governance perimeter with citation trails on every output.

What are common failure modes for AI agents?

Agents can get stuck in loops, take ineffective actions, misinterpret results, or pursue a goal that was poorly defined. They can also exceed their intended scope if not properly constrained. This is why production agents need guardrails, human review checkpoints, and clear task boundaries.